Entity Framework Core in ASP.NET Core Web API with SQL Server

Back to: ASP.NET Core Web API Tutorials

Entity Framework Core in ASP.NET Core Web API with SQL Server

Entity Framework Core (EF Core) is Microsoft's **Modern, Lightweight, and Cross-Platform Object-Relational Mapper (ORM)** for .NET applications. It allows developers to work with databases using .NET objects, eliminating most of the manual SQL writing while still giving full control over queries when needed.

When combined with **ASP.NET Core Web API** and **SQL Server**, EF Core provides a clean, scalable, and testable data access layer, essential for building real-time production APIs.

Why Use EF Core in ASP.NET Core Web API?

In simple terms, Entity Framework Core (EF Core) is Microsoft's modern, lightweight, cross-platform ORM that lets us work with a database using .NET classes, not raw SQL most of the time. It supports LINQ queries, change tracking, and migrations, and integrates deeply with ASP.NET Core's DI, logging, and configuration.

Key reasons:

- **Productivity Boost**: Automatically maps our C# classes (entities) to database tables. It eliminates repetitive SQL queries for common operations like insert, update, or delete.
- **Cross-Platform & Lightweight**: Runs on Windows, Linux, and macOS. Supports multiple databases like SQL Server, PostgreSQL, MySQL, SQLite, and more.
- **Code First Development**: Define database structure directly from our C# model classes. EF Core generates and maintains the database schema using **migrations**.
- **LINQ Support**: We can query our data using C# LINQ expressions instead of raw SQL, safer and more maintainable.
- **Change Tracking & Transactions**: EF Core automatically tracks entity states (Added, Modified, Deleted) and also handles database transactions efficiently.
- **Unit testing friendly:** Replace the real provider with in-memory or SQLite for tests; mock repositories/services around DbContext.
- **Performance:** Modern change tracker, compiled models, AsNoTracking, batch ops, and provider-specific optimizations.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

So, EF Core fits naturally into ASP.NET Core Web APIs, making it easy to create modern, scalable RESTful services that interact with relational databases like SQL Server, MySQL, Oracle, etc.

Which EF Core Packages to Install?

When working with EF Core in **ASP.NET Core Web API** with **SQL Server Database**, we must install the following NuGet packages:

1. **Microsoft.EntityFrameworkCore** → The core package that provides EF Core functionalities such as LINQ, Change Tracking, and Model Configuration.
2. **Microsoft.EntityFrameworkCore.SqlServer** → The Database Provider for SQL Server, adds SQL Server-specific features and optimizations.
3. **Microsoft.EntityFrameworkCore.Tools** → Provides PowerShell commands for database Migrations, Scaffolding, and Database Updates.

What is a Database Provider in EF Core?

Entity Framework Core (EF Core) is designed to be **Database-Agnostic**, meaning it can work with multiple database systems, such as SQL Server, MySQL, PostgreSQL, Oracle, or SQLite, without changing our codebase much.

But EF Core doesn't know how to communicate with these databases by default. That's where a **Database Provider Package** comes into play. A **Database Provider** is a special adapter or bridge that allows EF Core to translate our C# LINQ queries and entity operations into **Database-Specific SQL Commands** that the target database understands.

Each provider knows:

- The SQL Syntax of its database (T-SQL, MySQL SQL, PostgreSQL syntax, etc.).
- How to map .NET data types to database data types.
- How to manage transactions, constraints, sequences, and schema creation.
- How to optimize queries and handle specific database features.

So, **the Database Provider is the layer that connects our EF Core model with our chosen relational database**. Different databases (SQL Server, PostgreSQL, SQLite, etc.) need different providers.

Why Do We Need a Database Provider Package?

Let's consider an example. When you write this in EF Core:

- **var products = context.Products.Where(p => p.Price > 1000).ToList();**

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

EF Core itself cannot execute this query directly. It delegates this task to the installed **provider**. If you are using SQL Server, the provider translates the above LINQ query into something like:

```
SELECT [p].[Id], [p].[Name], [p].[Price], [p].[Stock]
FROM [Products] AS [p]
WHERE [p].[Price] > 1000;
```

If you switch to PostgreSQL, the same query might be generated slightly differently as follows:

```
SELECT p."Id", p."Name", p."Price", p."Stock"
FROM "Products" AS p
WHERE p."Price" > 1000;
```

So, **each provider ensures that EF Core behaves correctly according to the target database's syntax and capabilities**.

Commonly Used EF Core Database Providers

The following are some of the commonly used database providers in Entity Framework Core. Each of the following packages must be installed separately, depending on your database system.

- **Microsoft.EntityFrameworkCore.SqlServer → for SQL Server**
- **Pomelo.EntityFrameworkCore.MySql → for MySQL**
- **Npgsql.EntityFrameworkCore.PostgreSQL → for PostgreSQL**
- **Microsoft.EntityFrameworkCore.Sqlite → for SQLite**
- **Oracle.EntityFrameworkCore → for Oracle**

So, the **Database Provider Package** is the essential layer that gives EF Core its ability to communicate with any relational database. Without it, EF Core cannot function because it wouldn't know:

- What SQL syntax to use,
- How to translate LINQ,
- How to handle schema creation or migrations.

So, when we say *"**EF Core works with SQL Server**"*, what we really mean is *"**EF Core uses the SQL Server Provider to translate our .NET code into SQL Server-specific operations**."*

Entity Framework Core Tools

Now that EF Core can talk to our database (thanks to the provider), we need another important layer, **Tools**, to manage and evolve the database schema over time.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

What Are EF Core Tools?

EF Core Tools are **Command-Line Utilities** that allow developers to:

- Create database schemas
- Manage migrations
- Scaffold entity models from existing databases
- Update the database when our model changes

In simpler words, **EF Core Tools are the "bridge" between model classes and the actual physical database.** They don't run as part of your app; instead, we use them during **Development Time** to create, update, and maintain our database schema.

Understanding EF Core with the Code First Approach:

In EF Core, there are **Two Major Approaches** to creating and managing databases:

- **Database First** → We start with an existing database, and EF Core generates models (entities) from that database using scaffolding.
- **Code First** → We start by writing C# classes (our models), and EF Core automatically creates the database schema based on those classes.

In this example, we are focusing on the **Code First approach**, which is the most **Modern and Developer-Friendly** pattern.

Overview of the Product Management API

Before jumping into coding, let's visualize what we are building. We are going to create a simple **Product Management API** using **ASP.NET Core Web API**, **Entity Framework Core**, and **SQL Server**. The application's goal is to manage products, where we will be able to:

- Create new products
- Read product details
- Update product information
- Delete products

Step 1: Creating a New ASP.NET Core Web API Project

We will start by creating a new ASP.NET Core Web API project that will eventually use EF Core for database operations. So, create an a new ASP.NET Core Web API Project and named it **ProductManagementAPI**. Once you

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

created the project, you can safely delete or ignore the default **WeatherForecastController.cs** and **WeatherForecast.cs** since we will add our own controllers for the Product API.

Step 2: Installing Required EF Core Packages

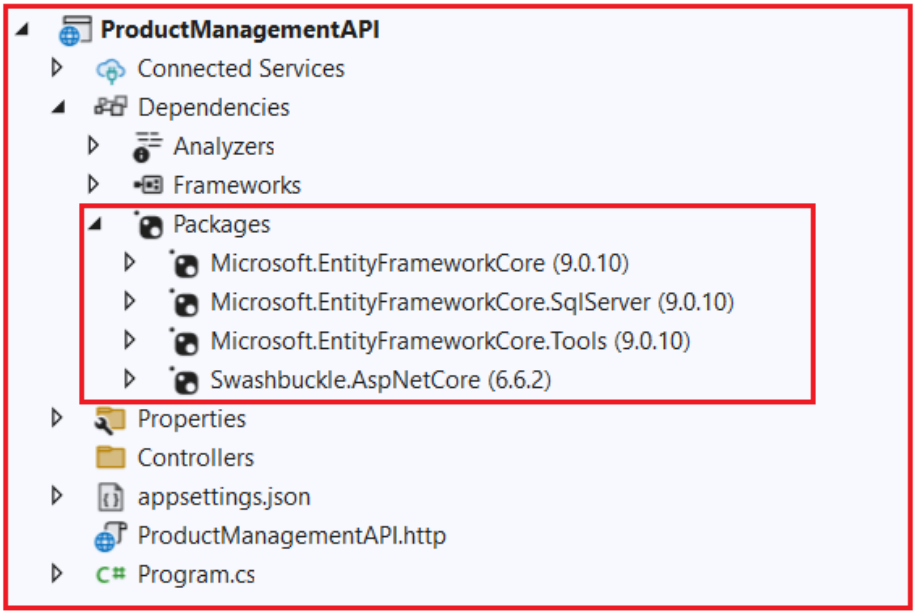
As we are going to work with EF Core and SQL Server, let us first install the required packages. We'll need the following packages:

- **Microsoft.EntityFrameworkCore** → Core functionality of EF Core.
- **Microsoft.EntityFrameworkCore.SqlServer** → Database provider for SQL Server.
- **Microsoft.EntityFrameworkCore.Tools** → Tools for migration and scaffolding.

You can install these packages using NuGet Package Manager solution or by executing the following command in the Visual Studio Package Manager Console (PMC):

- **Install-Package Microsoft.EntityFrameworkCore**
- **Install-Package Microsoft.EntityFrameworkCore.SqlServer**
- **Install-Package Microsoft.EntityFrameworkCore.Tools**

Once the above packages are installed, you can verify the same in the Dependencies/packages folder as shown in the below image:



Step 3: Creating the Product Entity Class

Now we will create the **Product entity**, which represents the database table we will eventually create through migrations. Create a new folder named **Models** (if not already present) and then add a class file named **Product.cs** inside it and copy-paste the following code.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
using System.ComponentModel.DataAnnotations.Schema;
namespace ProductManagementAPI.Models
{
    public class Product
    {
        public int Id { get; set; }                // Primary key
        public string Name { get; set; } = null!;  // Product name

        [Column(TypeName ="decimal(18,2)")]
        public decimal Price { get; set; }         // Product price
        public int Stock { get; set; }             // Available stock quantity
        public string? Description { get; set; }   // Optional description field
    }
}
```

What is the DbContext Class in Entity Framework Core?

In **Entity Framework Core (EF Core)**, the DbContext class acts as the **primary bridge between our .NET application and the database**. It belongs to the **Microsoft.EntityFrameworkCore** namespace. You can imagine it like a **session** or a **workspace** where all database interactions happen, such as:

- Manages the database connection,
- Fetching data (queries),
- Adding or updating data,
- Deleting records,
- Tracking entity changes,
- Executing transactions, and
- Applying migrations.

EF Core is an **Object-Relational Mapper (ORM)**. It converts our C# objects (entities) into database rows and vice versa. To do that, it needs a middle layer that knows:

- What entities (tables) exist,
- How to connect to the database,
- How to translate LINQ expressions into SQL queries, and
- How to track which objects have changed.

That middle layer is the **DbContext class**. In simple terms, DbContext is the brain of EF Core that keeps track of what you are doing with your entities and pushes those changes to the database when you call SaveChanges(). So, every time your application needs to talk to the database, it does so through a DbContext instance.

What Are the Tasks Performed by DbContext in Entity Framework Core?

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

The DbContext class performs several **key responsibilities** that make EF Core work smoothly. Let's explore them in depth:

Managing the Database Connection

- The DbContext opens and closes the connection to the database as needed.
- You don't have to manually manage connection strings, open connections, or execute commands.
- When a LINQ query is executed or data is saved, the DbContext automatically opens the connection, runs the query, and closes it afterward (unless explicitly told to keep it open).

Change Tracking

- DbContext keeps track of all entities that you load, add, update, or delete.
- It maintains an **internal state** for each entity, such as:
  - Added → new object to be inserted
  - Modified → existing object with changes
  - Deleted → object to be removed
  - Unchanged → no changes made
- This mechanism helps EF Core generate the correct SQL commands (INSERT, UPDATE, DELETE) during SaveChanges().

Querying Data

- DbContext allows you to use **LINQ (Language Integrated Query)** to query entities.
- LINQ queries are automatically translated into SQL commands by EF Core's query provider.
- You can fetch single entities, lists, or even perform joins and projections, all in C# syntax.

Saving Data

- After you make changes to your entity objects (add, edit, delete), DbContext tracks those changes.
- When you call **SaveChanges() or SaveChangesAsync(),** EF Core compiles all pending changes into appropriate SQL commands and executes them in one transaction.
- This ensures **atomicity**, either all changes succeed, or all are rolled back.

Transaction Management

- DbContext automatically wraps SaveChanges() inside a transaction.
- You can also create explicit transactions for advanced scenarios using: using var transaction = context.Database.BeginTransaction();
- This ensures that multiple operations are treated as one atomic unit.

Caching and Tracking Query Results

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- DbContext caches entities it retrieves during its lifetime. So, if you query the same entity twice within one context instance, EF Core fetches it from memory instead of querying the database again.
- This improves performance and consistency.

How Do We Create a DbContext Class in Entity Framework Core?

Creating a DbContext in EF Core is straightforward, but it's important to understand what each part does. You usually create your own class that **inherits from the base class DbContext** and defines your entities as **DbSet<TEntity> properties**.

Step 1: Create a Folder

Inside your ASP.NET Core Web API project, create a folder named **Data** or **Persistence** (depending on naming conventions). I am creating the Data folder.

Step 2: Create the DbContext Class

Create a new class file named **AppDbContext.cs** inside the **Data** folder, and copy-paste the following code.

```
using Microsoft.EntityFrameworkCore;
using ProductManagementAPI.Models;

namespace ProductManagementAPI.Data
{
    public class AppDbContext : DbContext
    {
        // Constructor that accepts DbContextOptions
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }

        // Define DbSets for your entities
        public DbSet<Product> Products { get; set; }
    }
}
```

Code Explanations:

Inheritance from DbContext

Our custom class (e.g., AppDbContext) inherits from Microsoft.EntityFrameworkCore.DbContext, which gives it all database-handling capabilities.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Constructor

The constructor accepts DbContextOptions<AppDbContext> which contains configuration like:

- Connection string
- Database provider (SQL Server, MySQL, etc.)
- Logging, lazy loading, etc.

These options are passed from the Program.cs file during dependency injection setup.

DbSet<TEntity> Properties

Each DbSet<TEntity> represents a **table** in the database.

- DbSet<Product> → corresponds to the Products table.
- Each entity type we define in DbSet is mapped to a database table during migrations.

What Are the Methods Provided by DbContext in EF Core?

The DbContext class provides several **methods** that make it powerful and flexible. Let's go through the most essential ones.

SaveChanges()

- Commits all tracked changes (Added, Modified, Deleted) to the database.
- EF Core automatically generates the necessary SQL statements and executes them in a transaction.
- Example: context.SaveChanges();

SaveChangesAsync()

- Asynchronous version of SaveChanges().
- Ideal for Web API applications to avoid blocking threads.
- Example: await context.SaveChangesAsync();

Add(), AddRange()

- Marks entities as "Added", so EF Core inserts them into the database when SaveChanges() is called.
- Example: context.Products.Add(new Product { Name = "Laptop", Price = 80000 });

Update(), UpdateRange()

- Marks existing entities as "Modified", so EF Core issues an UPDATE command.
- Example: context.Products.Update(product);

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Remove(), RemoveRange()

- Marks entities for deletion.
- Example: context.Products.Remove(product);

Find()

- Quickly finds an entity by its primary key.
- First checks the context's local cache before hitting the database.
- Example: var product = context.Products.Find(1);

What is a Database Connection String in Entity Framework Core?

In **Entity Framework Core (EF Core)**, a **Database Connection String** is a **Configuration String** that contains all the details required for your application to establish a connection with a database server. Think of it as the **address, credentials, and instructions** EF Core needs to locate and talk to your database.

When your DbContext runs (for example, while fetching data or saving changes), EF Core uses this connection string to:

- Know **Which Server** to connect to (e.g., SQL Server on your local machine or cloud),
- Know **Which Database** to use (e.g., ProductDB),
- Know **How to authenticate** (Windows Authentication or SQL Authentication),
- And manage other connection-level settings (like pooling, timeouts, etc.).

Without it, EF Core doesn't know where to go. It's a small string but carries powerful information, for example: "**Server=.; Database=ProductDB; Trusted_Connection=True; MultipleActiveResultSets=True;**"

Let's break it down briefly:

- **Server=.;** → The dot (.) means the local SQL Server instance.
- **Database=ProductDB;** → Name of the database to connect to.
- **Trusted_Connection=True;** → Use Windows Authentication instead of username/password.
- **MultipleActiveResultSets=True;** → Allows multiple queries to run on one connection (used by EF Core internally).

So, in EF Core, when we call something like options.UseSqlServer(connectionString) inside your Program.cs, this connection string is what EF Core uses to communicate with SQL Server.

Storing Database Connection String in appsettings.json File:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Let us procced and store the connection string inside appsettings.json file. The appsettings.json file in ASP.NET Core acts like a **Central Configuration File** for our application. It's similar to web.config in older .NET Framework apps, but it's **JSON-based**, easier to manage, and supports hierarchical configurations.

Now, within the appsetings.json file, we need to add a section for the configuration string. So, please modify the appsetings.json file as follows:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection": "Server=LAPTOP-
6P5NK25R\\SQLSERVER2022DEV;Database=ProductDB;Trusted_Connection=True;TrustServerCe
  }
}
```

Understanding the Hierarchy

- "**ConnectionStrings**" → A logical grouping of all connection strings (you can have multiple).
- "**DefaultConnection**" → The name/key of this connection string (you can name it whatever you like).
- The actual string value → " **Server=LAPTOP-6P5NK25R\\SQLSERVER2022DEV; Database=ProductDB; Trusted_Connection=True; TrustServerCertificate=True;**".

How to Configure Database Connection String and DbContext?

Once the connection string is defined, we must **tell EF Core** how to use it. This is done by **registering the DbContext** and **configuring it to use SQL Server** inside the **Program class**. Let's understand this step in depth.

In the modern **ASP.NET Core Minimal Hosting Model (from .NET 6 onwards)**, everything is configured inside **Program.cs**. We no longer have Startup.cs, so the setup happens here. So, please modify the Program class as follows:

```
using Microsoft.EntityFrameworkCore;
using ProductManagementAPI.Data;

namespace ProductManagementAPI
{
    public class Program
    {
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);

            // Add services to the container.
            builder.Services.AddControllers()
            .AddJsonOptions(options =>
            {
                // This will use the property names as defined in the C# model
                options.JsonSerializerOptions.PropertyNamingPolicy = null;
            });

            builder.Services.AddEndpointsApiExplorer();
            builder.Services.AddSwaggerGen();

            builder.Services.AddDbContext<AppDbContext>(options =>

options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")

            var app = builder.Build();

            // Configure the HTTP request pipeline.
            if (app.Environment.IsDevelopment())
            {
                app.UseSwagger();
                app.UseSwaggerUI();
            }

            app.UseHttpsRedirection();

            app.UseAuthorization();

            app.MapControllers();

            app.Run();
        }
    }
}
```

What's Happening Internally

When we register AddDbContext<AppDbContext>(), a few important things occur under the hood:

1. The AppDbContext class is added to the **Dependency Injection (DI)** container.
2. ASP.NET Core automatically creates and manages its lifetime (Scoped by default, one instance per request).
3. EF Core is configured to use SQL Server as its **Database Provider** for AppDbContext.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

4. The connection string is retrieved dynamically from the appsettings.json file.

Now, whenever a controller or service in your project requires access to the database, you can simply **inject the DbContext** through the constructor:

What is Database Migration in EF Core?

In **Entity Framework Core (EF Core)**, **Database Migration** is the process of **updating your database schema** in sync with your application's data model, without losing the existing data. When we use the **Code First** approach, we define our data structure using **C# classes** (entities). Over time, these models might change, for example, we may:

- Add a New Property (column),
- Rename a Field,
- Remove a Column,
- Create a New Entity (table), or
- Modify a Relationship (like one-to-many or many-to-many).

If we change our entity classes, our **Database Structure** must also change to reflect those updates. Instead of manually writing SQL ALTER TABLE statements, EF Core automates this process through **Migrations**. Think of Migrations as a **Version Control System** for your database schema.

- When we first create our models, EF Core can generate an initial migration that builds our database from scratch.
- When we later modify our model classes, EF Core can generate incremental migrations that apply just the required schema changes, no full database recreation.
- We can always track, revert, or reapply migrations, just like Git commits for code.

So, in short, EF Core Migrations allow our database to evolve safely and predictably as our C# models evolve.

How to Create a Migration in EF Core?

To create a migration, EF Core provides commands via:

- **Package Manager Console (PMC)** in Visual Studio, or
- **.NET CLI** (Command Line Interface).

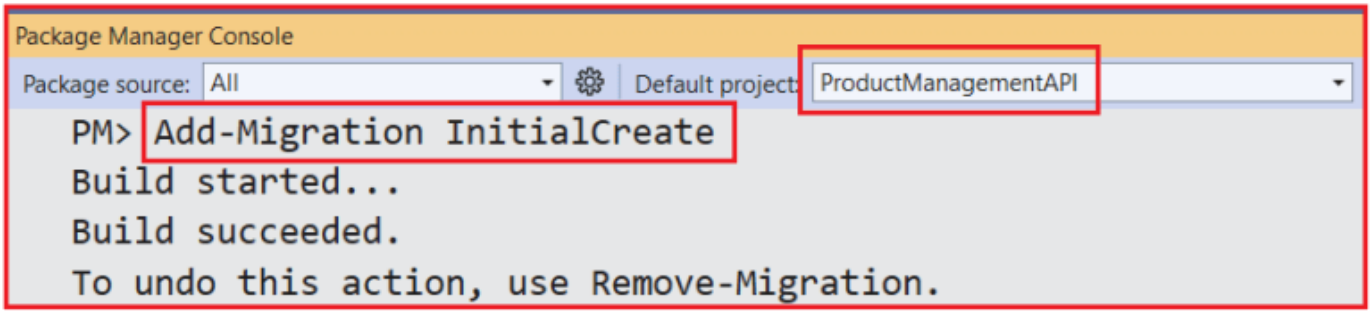
Open Package Manager Console

In Visual Studio, please select **Tools** → **NuGet Package Manager** → **Package Manager Console** which will open the PMC window.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Run the Migration Command

In the Package Manager Console, execute the **Add-Migration InitialCreate** command as shown in the below image. While executing the command, please select the project which contains the DbContext class.



Here:

- **Add-Migration** → Command to generate a migration file.
- **InitialCreate** → Descriptive name of the migration (you can use any meaningful name which you have not provided earlier).

What Happens Internally When You Run Add-Migration?

The moment we execute Add-Migration, EF Core performs the following steps behind the scenes:

1. **Compares the Current Model with the Last Applied Migration if any**
    - EF Core inspects your current DbContext and the entities defined within it.
    - It checks if there's any difference between the current model and the model snapshot from the previous migration (or none, if this is the first).
2. **Identifies Changes**
    - EF Core determines what new tables, columns, or constraints are needed.
3. **Generates Migration Files**
    - It then creates a new **Migration folder** (if not already present) in your project.
    - Inside, EF Core creates 2 files.
    - These files contain C# code representing the SQL operations (CREATE TABLE, ALTER COLUMN, etc.) that EF Core will execute.
4. **Keeps a Snapshot**
    - EF Core creates or updates a model snapshot file that represents the current state of your data model.
    - This helps EF Core understand what changed in future migrations.

In short, the Add-Migration command translates your C# model changes into database schema change scripts.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

What Are the Different Migration Files Generated When We Execute Add-Migration Command?

When you run Add-Migration, EF Core generates **two or three important files** under a new folder named **Migrations**.

Let's understand each in detail.

Migration Class File (e.g., 20251024xxxxxx_InitialCreate.cs)

This is the main file that defines **what changes to apply to the database**. It contains two important methods:

- **Up()** → Defines operations to **Apply** (e.g., Create Tables, Add Columns).
- **Down()** → Defines operations to **Revert** (e.g., Drop Tables, Remove Columns).

When Update-Database runs, EF Core executes the Up() method. If you ever need to roll back this migration, EF Core uses the Down() method.

Designer File (e.g., 20251024xxxxxx_InitialCreate.Designer.cs)

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

This file is automatically generated and contains metadata information used by EF Core Tools internally. It defines:

- The full model structure (entities, relationships, indexes, constraints, etc.).
- How the model maps to the database.

You usually **don't edit this file manually**, as it's meant for EF Core's internal tracking.

Model Snapshot File (ModelSnapshot.cs)

This file resides in the same **Migrations** folder and acts as a **blueprint of your model's current state**. EF Core uses this snapshot later to:

- Compare your updated entity classes,
- Detect new schema changes,
- Generate the next migration accurately.

What is Update-Database Command and Why Is It Important in EF Core Migration?

Once a migration is created, it only exists as a **C# file**, your database hasn't been created/updated yet. The Update-Database command is what actually **executes the migration code** (the **Up() method**) and applies those changes to your physical database.

Think of **Add-Migration** as **writing the script** and **Update-Database** as **running the script** on your real database. So, until you run Update-Database command, your SQL Server database structure doesn't change, even though you have the migration file ready.

What Happens When You Execute Update-Database Command?

When we run:

- **Update-Database**

EF Core goes through a series of steps internally. Let's walk through them one by one.

Step 1: Validates Context and Migration Files

EF Core first verifies that:

- The current DbContext is valid and can connect to the configured database.
- The migration files exist and are in sync with the model snapshot.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Step 2: Connects to the Database

Using the connection string defined in your appsettings.json, EF Core establishes a connection to the target database (e.g., ProductDB on SQL Server). If the database doesn't exist, EF Core can **create it automatically**.

Step 3: Checks Migration History

EF Core inspects a special internal table named: **__EFMigrationsHistory**. This table contains a list of all migrations that have already been applied something like the below. EF Core uses this to determine which migrations are new and need to be applied.

Step 4: Executes the Up() Method

For every Pending Migration, EF Core:

- Translates the C# migration code into SQL commands.
- Executes those SQL commands on your database.
- Commits the changes inside a transaction.

Step 5: Updates the Migration History Table

After successfully applying the migration, EF Core adds a new record to the **__EFMigrationsHistory** table, marking that migration as applied. This prevents EF Core from applying it again in future runs.

Step 6: Final Confirmation

After the migration is applied successfully:

- The database schema now matches your entity model.
- You can verify the new tables in **SQL Server Management Studio (SSMS)**.
- Your application is now ready to query or insert data using EF Core.

Applying the Migration:

Now, please open the Package Manager Console and execute the Update-Database command as shown in the below image:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Verifying the Database:

Now, please verify the database and you should see the **ProductDB** database with the **Products** and **__EFMigrationsHistory database tables as shown in the image below.**

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

What are CRUD Operations?

CRUD stands for the **Four Fundamental Operations** that can be performed on persistent data in a database:

- **C** → Create (Insert new data)
- **R** → Read (Retrieve data)
- **U** → Update (Modify existing data)
- **D** → Delete (Remove data)

These four operations form the backbone of almost every web application, regardless of its domain, whether it's product management, user management, booking systems, or inventory systems. Imagine a **Product Management System** that maintains information about different products. Here's how each CRUD operation plays a role:

- **Create:** Adding a new product (e.g., "Laptop") to the Products table.
- **Read:** Retrieving a list of all products or a specific product by ID.
- **Update:** Editing the details of an existing product (like changing its price).
- **Delete:** Removing a product that is no longer available.

These operations collectively ensure that your system can maintain and manipulate data seamlessly over time.

How Do We Perform CRUD Operations in Entity Framework Core (EF Core)?

In EF Core, all CRUD operations are performed through the **DbContext class**. The DbContext tracks entities in memory and interacts with the database through DbSet<TEntity> collections. Let's understand the conceptual flow for each operation before we move to the actual controller.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Create (Insert)

- You create a new entity object (e.g., a new Product instance).
- You add it to the corresponding DbSet (_context.Products.Add(product)).
- EF Core marks it as “Added”.
- When you call _context.SaveChanges(), EF Core generates an INSERT SQL command and executes it.

Read (Retrieve)

- You use LINQ queries like .ToList(), .Find(), or .Where() to fetch data.
- EF Core translates the LINQ expressions into SQL SELECT queries and maps the results back into C# objects.

Update (Modify)

- You fetch an existing entity from the database.
- You modify one or more of its property values.
- EF Core marks it as “Modified”.
- When you call _context.SaveChanges(), EF Core generates the appropriate UPDATE SQL statement.

Delete (Remove)

- You fetch the entity to be deleted.
- You call _context.Remove(entity) or _context.Products.Remove(entity).
- EF Core marks it as “Deleted”.
- On calling _context.SaveChanges(), EF Core executes a DELETE SQL statement.

Create a Product API Controller and Perform CRUD Operations

Let’s now put theory into practice by creating a **Product API Controller** that will perform all CRUD operations using **Entity Framework Core** and our **Product entity**. So, create a new Empty API Controller named **ProductsController** within the **Controllers** folder and the add following code into it.

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using ProductManagementAPI.Data;
using ProductManagementAPI.Models;

namespace ProductManagementAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductsController : ControllerBase
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    {
        private readonly AppDbContext _context;

        public ProductsController(AppDbContext context)
        {
            _context = context;
        }

        // GET: api/products
        // Retrieve all products
        [HttpGet]
        public async Task<ActionResult<List<Product>>> GetAll()
        {
            // Fetch all products from database
            var products = await _context.Products.ToListAsync();

            // Return HTTP 200 OK with data
            return Ok(products);
        }

        // GET: api/products/{id}
        // Retrieve product by ID
        [HttpGet("{id}")]
        public async Task<ActionResult<Product>> GetById(int id)
        {
            // Find product by primary key
            var product = await _context.Products.FindAsync(id);

            if (product == null)
                return NotFound(new { Message = $"Product with ID {id} not
found." });

            return Ok(product);
        }

        // POST: api/products
        // Create new product
        [HttpPost]
        public async Task<IActionResult> Create([FromBody] Product product)
        {
            if (product == null)
                return BadRequest(new { Message = "Invalid product data." });

            _context.Products.Add(product);
            await _context.SaveChangesAsync();

            // Return 201 Created with location header
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
            return CreatedAtAction(nameof(GetById), new { id = product.Id },
product);
        }

        // PUT: api/products/{id}
        // Update existing product
        [HttpPut("{id}")]
        public async Task<IActionResult> Update(int id, [FromBody] Product
updatedProduct)
        {
            if (id != updatedProduct.Id)
                return BadRequest(new { Message = "Product ID mismatch." });

            // Check if product exists
            var product = await _context.Products.FindAsync(id);
            if (product == null)
                return NotFound(new { Message = $"Product with ID {id} not
found." });

            // Update fields
            product.Name = updatedProduct.Name;
            product.Price = updatedProduct.Price;
            product.Stock = updatedProduct.Stock;
            product.Description = updatedProduct.Description;

            _context.Entry(product).State = EntityState.Modified;
            await _context.SaveChangesAsync();

            return Ok(new { Message = "Product updated successfully.", Product =
product });
        }

        // DELETE: api/products/{id}
        // Delete existing product
        [HttpDelete("{id}")]
        public async Task<IActionResult> Delete(int id)
        {
            var product = await _context.Products.FindAsync(id);
            if (product == null)
                return NotFound(new { Message = $"Product with ID {id} not
found." });

            _context.Products.Remove(product);
            await _context.SaveChangesAsync();

            return Ok(new { Message = "Product deleted successfully." });
        }
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    }
}
```

What is EF Core Logging?

Entity Framework Core (EF Core) Logging is a built-in mechanism that allows developers to **view and monitor the internal database operations** performed by EF Core during runtime, such as SQL queries, parameters, transactions, database connections, and errors.

Whenever your application interacts with the database (for example, while inserting, updating, deleting, or retrieving data), EF Core automatically generates corresponding **SQL statements**. EF Core Logging helps you *see* these SQL statements in real-time through console output, or log files.

Logging helps you understand:

- What exact SQL statements EF Core executes.
- How parameters are passed to those SQL commands.
- How long each query takes to run.
- Whether a transaction has started, committed, or rolled back.
- What went wrong when an exception or database error occurs.

Without logging, developers would have to **guess** what EF Core is doing behind the scenes. With logging enabled, we gain **transparency and control**, allowing for better debugging, optimization, and database performance tuning.

When to Enable EF Core Logging

Logging is especially useful during these scenarios:

- When developing or debugging CRUD operations.
- When optimizing database queries.
- When verifying migrations and SQL schema changes.
- When diagnosing errors such as “Invalid column name,” “Foreign key constraint,” or “Timeout expired.”
- During testing or QA, to trace real database interactions.

In production environments, however, **you should log selectively** (only errors or warnings) to avoid performance overhead and exposing sensitive SQL data.

How to Enable EF Core Logging

EF Core logging is highly flexible and integrates with **ASP.NET Core’s built-in logging system** (Microsoft.Extensions.Logging). You can log directly to:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- The **Console Output** (ideal during development),
- The **Debug Window** in Visual Studio,
- Or even to structured logging frameworks like **Serilog**, **NLog**, or **Application Insights**.

Let's explore the simplest and most practical configuration, **logging to the Console**. By default Logging is enabled in ASP.NET Core.

Testing Each CRUD Operation

Let's test one operation at a time, with full explanations of what happens behind the scenes and what you will see in EF Core logs.

CREATE → POST Request

Insert a new product into the database.

Method: POST
URL: https://localhost:7144/api/products
Request Body (JSON):

```
{
  "Name": "Wireless Mouse",
  "Price": 1200,
  "Stock": 50,
  "Description": "Ergonomic wireless mouse with Bluetooth support"
}
```

EF Core Logging Output

Your console window will show the following:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

UPDATE → PUT Request

Update an existing product's details.

Method: PUT

URL: https://localhost:7144/api/products/1

Request Body (JSON):

```
{
  "Id": 1,
  "Name": "Wireless Mouse Pro",
  "Price": 1500,
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    "Stock": 60,
    "Description": "Upgraded ergonomic Bluetooth mouse"
  }
```

EF Core Logging Output

Your console window will show the following:

READ ALL → GET Request

Retrieve all products from the database.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Method: GET

URL: https://localhost:7144/api/products

No body is required.

EF Core Logging Output

Your console window will show the following:

READ BY ID → GET Request

Retrieve a single product by its ID.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Method: GET

URL: https://localhost:7144/api/products/1

No body is required.

EF Core Logging Output

Your console window will show the following:

DELETE → DELETE Request

Delete a product from the database.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Method: DELETE

URL: https://localhost:7144/api/products/1

No body is required.

EF Core Logging Output

Your console window will show the following:

How EF Core Logging Helps During Testing

While you execute these requests, you notice that EF Core printing SQL logs in your console. This confirms that:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- EF Core is correctly translating your LINQ and DbContext actions into SQL queries.
- Your API and database connection are working perfectly.
- You can see exactly what parameters were sent, how long each query took, and how EF Core handled transactions.

This transparency ensures that your CRUD endpoints are not only working, but also working **efficiently and securely**.

Understanding SET IMPLICIT_TRANSACTIONS OFF and SET NOCOUNT ON:

Those two statements you saw in the EF Core console output:

- SET IMPLICIT_TRANSACTIONS OFF;
- SET NOCOUNT ON;

are *not* commands we explicitly wrote. EF Core adds them automatically before executing our actual SQL. Let us understand these two commands in more detail.

SET IMPLICIT_TRANSACTIONS OFF;

In SQL Server, there are two modes for handling transactions:

- **Explicit Transactions**: You start them manually using BEGIN TRANSACTION … COMMIT.
- **Implicit Transactions**: SQL Server automatically starts a new transaction for each DML operation (INSERT, UPDATE, DELETE, etc.) and COMMIT or ROLLBACK depend on the Transaction mode.

When **IMPLICIT_TRANSACTIONS** is **ON**, every DML statement automatically begins a transaction that stays open until you explicitly commit it.

When **IMPLICIT_TRANSACTIONS** is **OFF**, each individual DML command executes in **Auto Commit mode**. it's automatically committed once the command finishes successfully.

Why EF Core Uses SET IMPLICIT_TRANSACTIONS OFF

EF Core manages transactions **Explicitly**. It set IMPLICIT_TRANSACTIONS to OFF which gives EF Core full control, it can decide when to:

- Execute a command individually (Auto Commit Mode),
- Or wrap multiple statements in a transaction (when you call SaveChanges() with multiple operations, or use Database.BeginTransaction()).

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

In short: SET IMPLICIT_TRANSACTIONS OFF ensures EF Core stays in control of transaction boundaries and prevents SQL Server from silently holding open transactions.

SET NOCOUNT ON;

By default, after executing any DML operation, SQL Server sends back an extra message to the client like: (1 row(s) affected). This “row count” message isn’t part of the query result. It is an informational message. When you enable SET NOCOUNT ON, SQL Server suppresses those messages.

Why EF Core Uses SET NOCOUNT ON

These “(n rows affected)” messages may look harmless, but:

- They add unnecessary network traffic.
- They can interfere with tools that expect result sets instead of messages.
- EF Core (and ADO.NET) already knows the number of affected rows from the command’s metadata.

Therefore, EF Core tells SQL Server: “Don’t send me those (n rows affected) messages, I will handle result counts myself.” It’s a small but important optimization that improves efficiency, especially during bulk inserts or multiple DML statements in one transaction.

When to Generate Another Migration File?

In EF Core’s **Code First** approach, our model classes (entities) define the structure of the database.
When we change our entity classes, for example:

- Add a new property to an existing entity.
- Create a new entity class.
- Rename or delete a column.
- Modify relationships (like changing one-to-many to many-to-many).

The database schema in SQL Server no longer matches our updated model. EF Core must be informed about these changes so it can generate the corresponding **SQL schema modification scripts**. That’s exactly what migrations do.

The first migration creates all tables. Subsequent migrations record incremental changes as your model evolves. So, we generate **another migration file** whenever we **modify entity classes or DbContext** that affects the database structure. In simple terms, we generate another migration file when our model changes, but we want to keep our existing data while updating the database schema incrementally. Let us understand the use of regeneration Migration with an example.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Modify the Product Entity (Add New Properties)

We now decide to track who created the product and when it was added to inventory. For this, we need to add two new properties. So, modify the Product entity as follows:

```
using System.ComponentModel.DataAnnotations.Schema;
namespace ProductManagementAPI.Models
{
    public class Product
    {
        public int Id { get; set; }                // Primary key
        public string Name { get; set; } = null!;  // Product name

        [Column(TypeName ="decimal(18,2)")]
        public decimal Price { get; set; }         // Product price
        public int Stock { get; set; }             // Available stock quantity
        public string? Description { get; set; }   // Optional description field

        // New properties added
        public string? CreatedBy { get; set; }     // Name of the user who added
the product
        public DateTime CreatedDate { get; set; }  // When the product was added
    }
}
```

With this, we just changed our **C# model**, but our database doesn't yet know about these new columns. Now, we need to tell EF Core to compare our current model with the last migration snapshot and record the differences. To do so, we need to generate a new Migration. So, open Visual Studio Package Manager Console (PMC) and execute the following command which will generate a new Migration.

- **Add-Migration AddAuditFieldsToProduct**

What Happens Internally

1. EF Core scans your AppDbContext and Product entity.
2. It compares this with the last migration snapshot.
3. It finds two new properties: CreatedBy and CreatedDate.
4. It creates a new migration file under the **Migrations** folder.

Now, imagine you realize you made a mistake. Perhaps the CreatedDate column should be **nullable** (because not every record will have it yet), but you forgot to set nullable: true. You haven't yet applied the migration (i.e., you didn't run Update-Database). To fix it, you can safely **remove the last migration**.

When to Use the Remove-Migration Command

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Sometimes, after running Add-Migration, you realize:

- You made a mistake in your model,
- The migration generated unintended schema changes,
- You want to modify the migration name or combine small changes into one migration.

In those cases, you need to remove that **latest (unapplied)** migration safely, without affecting existing applied migrations. So, execute the following command in the Package Manager Console:

- Remove-Migration

The Remove-Migration command simply **deletes the most recently added migration** and rolls back its snapshot if it hasn't yet been applied to the database. Once the above command executed successfully, you will see the following result:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

What Happens Internally

1. EF Core deletes the most recent migration file from the Migrations folder.
2. It restores the previous state of the ModelSnapshot file.
3. No database changes are made (since you didn't apply it yet).

Then, Correct Your Model

Modify your entity again as follows:

```
using System.ComponentModel.DataAnnotations.Schema;
namespace ProductManagementAPI.Models
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
{
    public class Product
    {
        public int Id { get; set; }                // Primary key
        public string Name { get; set; } = null!;  // Product name

        [Column(TypeName ="decimal(18,2)")]
        public decimal Price { get; set; }         // Product price
        public int Stock { get; set; }             // Available stock quantity
        public string? Description { get; set; }   // Optional description field

        // New properties added
        public string? CreatedBy { get; set; }     // Name of the user who added
the product
        public DateTime? CreatedDate { get; set; }  // When the product was added
    }
}
```

Generate the migration again and apply the Migration by executing the following command in PMC:

- **Add-Migration AddAuditFieldsToProduct**
- **Update-Database**

Now, your corrected migration is applied and you can verify the same in the database as shown in the below image.

Note: If you verify the **__EFMigrationsHistory** table, then you will see another entry for the **AddAuditFieldsToProduct**.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

In Short

- **Add-Migration** → Generate a new migration file after model changes.
- **Remove-Migration** → Safely delete the last migration (if not yet applied).

Each command plays a vital role in maintaining synchronization between your **EF Core model**, **migration history**, and the **actual SQL Server schema**, ensuring your Code-First Web API evolves cleanly and predictably over time.

What happens if Migration is Already Applied?

If you already ran Update-Database, you can't remove it directly, EF Core will warn you. In that case, first roll back to the previous migration. Now, we have applied the Migration by executing the Update-Database command. Let us try to execute the Remove-Migration command in Visual Studio PMC and see what happens.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

As you can see in the above, we are getting an error. In such situations, we first need to Roll back our database to its previous Migration and then we need to Remove Migration.

Update-Database to Previous Migration:

Our previous Migration name is InitialCreate. So, please execute the following command in the PMC.

- **Update-Database InitialCreate**

The above will roll back our database to InitialCreate Migration. Now, if you check the database, then you will see, newly added columns are no longer exists as shown in the below image.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Note: If you verify the **__EFMigrationsHistory** table, then you will see the entry for the **AddAuditFieldsToProduct** is removed.

Remove the Latest Migration:

Then remove the unwanted one by executing the following command in PMV:

- **Remove-Migration**

This keeps your **code and database perfectly aligned**.

Entity Framework Core seamlessly integrates with ASP.NET Core Web API and SQL Server to simplify data access, automate schema management, and reduce boilerplate code. By combining the Code First approach, migrations, and logging, developers can build robust, scalable, and maintainable APIs with clear insight into database operations and schema evolution.

Registration Open – Mastering Design Patterns, Principles, and Architectures using .NET

New Batch Starts: 11th March, 2026

Session Time: 6:30 AM – 08:00 AM IST

Advance your career with our expert-led, hands-on live training program. Get complete course details, the syllabus, and Zoom credentials for demo sessions via the links below.

View Course Details & Get Demo Credentials

Registration Form

Join Telegram Group

Join WhatsApp Group

Contact: +91 70218 01173 (Call / WhatsApp)

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent | Do Not Sell My Information

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF